

SECTION 37 — FINAL REVISION SHEETS

Purpose: Ultra-dense, scannable one-page reference sheets for last-minute review before a Senior SDET & QA Architect interview (Deloitte, FAANG, Big 4). Each sheet is designed to be read in under 10 minutes and covers the most frequently tested, most commonly confused, and most senior-differentiating concepts in each domain.

37.1 One-Page Java Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
<code>volatile</code>	Guarantees visibility across threads; reads/writes go directly to main memory	Does NOT guarantee atomicity — <code>volatile int i; i++</code> is still a race condition (read-modify-write)
<code>synchronized</code>	Mutual exclusion via intrinsic/monitor lock on object or class	<code>synchronized</code> on non- <code>static</code> method locks <code>this</code> ; on <code>static</code> method locks the <code>Class</code> object — two different locks
<code>ReentrantLock</code>	Explicit lock with <code>tryLock()</code> , fairness, condition variables	Must call <code>unlock()</code> in <code>finally</code> block — forgetting it causes deadlock on exception
<code>ThreadLocal<T></code>	Per-thread variable storage, no sharing	Memory leak in thread pools (Tomcat, etc.) — always <code>remove()</code> in <code>finally</code>
<code>CompletableFuture</code>	Async composition pipeline with <code>thenApply</code> , <code>thenCompose</code> , <code>exceptionally</code>	<code>thenApply</code> maps synchronously on completing thread; <code>thenApplyAsync</code> spawns new task
<code>equals()</code> / <code>hashCode()</code> contract	If <code>a.equals(b)</code> then <code>a.hashCode() == b.hashCode()</code> must hold	Break contract → objects lost in <code>HashMap</code> ; equal objects in different buckets
Generics erasure	Generic type params erased at compile time; <code>List<String></code> becomes <code>List</code> at runtime	Cannot do <code>instanceof List<String></code> or create <code>new T[]</code> — use wildcard or <code>Class<T></code> token
<code>final</code> keyword	On class: no subclass. On method: no override. On variable: no reassignment	<code>final</code> object reference can't change, but object's mutable fields CAN be changed
<code>static</code> initializer	Runs once when class is first loaded by <code>ClassLoader</code>	Exceptions thrown here become <code>ExceptionInInitializerError</code> , class unusable
Checked vs Unchecked	Checked: must declare/catch (compile time). Unchecked: extends <code>RuntimeException</code>	<code>Error</code> is not <code>Exception</code> — catch <code>Throwable</code> only in framework/thread boundary code
String interning	String literals go to string pool; <code>new String("x")</code> creates heap object	<code>"a" == "a"</code> is <code>true</code> (pool); <code>new String("a") == new String("a")</code> is <code>false</code>
<code>Optional<T></code>	Container for potentially null value; avoids <code>NullPointerException</code>	<code>Optional.get()</code> without <code>isPresent()</code> throws <code>NoSuchElementException</code> — use <code>orElse</code> / <code>map</code>
Functional interfaces	Interface with exactly one abstract method; usable as lambda target	<code>@FunctionalInterface</code> annotation is optional but documents intent and triggers compiler check
Stream laziness	Intermediate ops (<code>filter</code> , <code>map</code>) are lazy; terminal op (<code>collect</code> , <code>forEach</code>) triggers pipeline	Calling a terminal op on an already-consumed stream throws <code>IllegalStateException</code>
<code>instanceof</code> pattern	Java 16+: <code>if (obj instanceof String s)</code> binds <code>s</code> in scope	<code>s</code> is only in scope in the <code>true</code> branch; using it in <code>else</code> is compile error

CRITICAL CODE PATTERNS

```
// 1. Thread-safe singleton with double-checked locking
public class Singleton {
```

```

private static volatile Singleton instance; // volatile is MANDATORY

public static Singleton getInstance() {
    if (instance == null) { // first check (no lock)
        synchronized (Singleton.class) {
            if (instance == null) { // second check (with lock)
                instance = new Singleton();
            }
        }
    }
    return instance;
}

// 2. CompletableFuture async pipeline
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> fetchData()) // runs in ForkJoinPool.commonPool()
    .thenApply(data -> transform(data)) // sync on completing thread
    .thenApplyAsync(data -> enrich(data)) // async on common pool
    .exceptionally(ex -> "fallback"); // handles any upstream exception

// 3. ThreadLocal with cleanup
private static final ThreadLocal<DateFormat> formatter =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

try {
    return formatter.get().format(date);
} finally {
    formatter.remove(); // CRITICAL in thread pools
}

// 4. Functional interface + method reference
List<String> sorted = names.stream()
    .filter(Objects::nonNull)
    .map(String::toUpperCase)
    .sorted(Comparator.comparing(String::length).reversed())
    .collect(Collectors.toList());

// 5. Custom exception with context
public class TestExecutionException extends RuntimeException {
    private final String testId;
    private final int stepNumber;

    public TestExecutionException(String message, String testId, int step, Throwable cause) {
        super(message, cause);
        this.testId = testId;
        this.stepNumber = step;
    }
    // getters...
}

```

COMPARISON TABLES

Feature	synchronized	ReentrantLock
Syntax	keyword	explicit API
Try-lock	✗	tryLock(timeout) ✓
Fairness	✗	configurable ✓
Multiple conditions	✗	newCondition() ✓
Auto-release	✓ (block exit)	✗ (must call unlock())
Interruptible wait	✗	lockInterruptibly() ✓

Stream Terminal Op	Returns	Triggers Pipeline?
collect()	Collection/T	✓

Stream Terminal Op	Returns	Triggers Pipeline?
<code>forEach()</code>	void	✓
<code>count()</code>	long	✓
<code>findFirst()</code>	Optional	✓ (short-circuits)
<code>anyMatch()</code>	boolean	✓ (short-circuits)
<code>toList()</code> (Java 16+)	unmodifiable List	✓

INTERVIEW TRAP ANSWERS

- **"Is `HashMap` thread-safe?"** — No. Use `ConcurrentHashMap`. `Collections.synchronizedMap()` synchronizes on the whole map (coarse); `ConcurrentHashMap` uses segment/bucket-level locking (fine-grained). `synchronizedMap` is still not safe for compound operations like check-then-act.
- **"What happens if you override `equals()` but not `hashCode()`?"** — Objects that are `.equals()` may land in different buckets → `HashMap.get()` returns `null` even though the key logically exists. Your set can contain "duplicates."
- **"String is immutable — why?"** — Security (class loading by name), string pool efficiency, thread-safety, `hashCode` caching. The `char[]` / `byte[]` backing array is `private final` and never exposed.
- **"Can `final` variables be changed?"** — The reference cannot be reassigned, but if the object is mutable, its fields can change. `final List<String> list = new ArrayList<>(); list.add("x");` — perfectly legal.
- **"When does `static` initializer run?"** — First time the class is loaded. If it throws, class is permanently broken with `ExceptionInInitializerError`.
- **"What is a memory barrier?"** — `volatile` write inserts a `StoreLoad` memory barrier; ensures all preceding writes are flushed before the volatile write, and all subsequent reads see fresh values. This is what makes double-checked locking safe with `volatile`.

ARCHITECTURE BOUNDARY

```

JVM Runtime
├─ ClassLoader Subsystem
│   └─ Loads class → triggers static initializers
├─ Heap
│   ├── Young Gen (Eden + S0/S1) ← new objects
│   └─ Old Gen ← survivors after GC cycles
├─ Metaspace ← class metadata, static fields (Java 8+)
├─ Stack (per-thread) ← local vars, method frames, ThreadLocal storage
└─ Thread Scheduler
    ├── volatile: forces Main Memory read/write (bypasses CPU cache)
    └─ synchronized/Lock: enforces mutual exclusion via monitor
  
```

37.2 One-Page Collections Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
<code>ArrayList</code>	Dynamic array; $O(1)$ amortized add; $O(n)$ insert/remove middle	Grows by 50% capacity; no thread-safety; use <code>CopyOnWriteArrayList</code> for concurrent reads
<code>LinkedList</code>	Doubly-linked list; $O(1)$ add/remove head/tail; $O(n)$ index access	High memory overhead (node objects + pointers); poor cache locality vs <code>ArrayList</code>
<code>HashMap</code>	Hash table: array of buckets + linked list/tree chains	Load factor 0.75 default → resize at 75% full; Java 8+ treeifies bucket at 8 nodes
<code>TreeMap</code>	Red-black BST; $O(\log n)$ all ops; sorted by natural order or <code>Comparator</code>	Keys must be <code>Comparable</code> or provide <code>Comparator</code> ; <code>null</code> key throws <code>NullPointerException</code>
<code>LinkedHashMap</code>	<code>HashMap</code> + doubly-linked list; preserves insertion order (or access order)	Access-order mode with <code>removeEldestEntry()</code> → perfect LRU cache implementation

Concept	One-Line Definition	Critical Trap
<code>HashSet</code>	Backed by <code>HashMap</code> ; one <code>null</code> allowed; $O(1)$ add/contains/remove	Order not guaranteed; use <code>LinkedHashSet</code> for insertion order, <code>TreeSet</code> for sorted
<code>PriorityQueue</code>	Min-heap by default; $O(\log n)$ offer/poll; $O(n)$ contains/remove	NOT thread-safe; use <code>PriorityBlockingQueue</code> for producer-consumer scenarios
<code>ArrayDeque</code>	Resizable circular array; faster than <code>LinkedList</code> for stack/queue	No <code>null</code> elements; preferred over <code>Stack</code> (synchronized) for LIFO use cases
<code>ConcurrentHashMap</code>	Thread-safe; fine-grained locking per segment (Java 7) / bucket CAS (Java 8+)	<code>putIfAbsent()</code> is atomic; but <code>get() + put()</code> combo is NOT — use <code>computeIfAbsent()</code>
<code>CopyOnWriteArrayList</code>	Creates new array copy on every write; reads are lock-free	Expensive for write-heavy workloads; ideal for read-heavy, rarely-modified lists
<code>Collections.unmodifiableList()</code>	Wraps list, throws <code>UnsupportedOperationException</code> on mutation	The underlying list is still mutable! Get a reference to it and you can still mutate it
<code>List.of()</code> / <code>Map.of()</code> (Java 9+)	Truly immutable; <code>null</code> elements/keys forbidden	Throws <code>NullPointerException</code> on <code>null</code> ; <code>UnsupportedOperationException</code> on add/set/remove
<code>Comparator.comparing()</code>	Builds type-safe comparator from key extractor	Chain with <code>.thenComparing()</code> for multi-field sort; <code>.reversed()</code> reverses entire chain
<code>Iterator.remove()</code>	Safe removal during iteration	Using <code>list.remove()</code> inside for-each throws <code>ConcurrentModificationException</code>
Fail-fast vs Fail-safe	Fail-fast iterators (<code>ArrayList</code> , <code>HashMap</code>) throw <code>CME</code> on structural mod; Fail-safe (<code>ConcurrentHashMap</code> , <code>CopyOnWriteArrayList</code>) iterate on snapshot	<code>CME</code> is best-effort — not guaranteed; don't rely on it for logic

CRITICAL CODE PATTERNS

```
// 1. LRU Cache using LinkedHashMap
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder=true
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

// 2. Thread-safe atomic computeIfAbsent
ConcurrentHashMap<String, List<String>> map = new ConcurrentHashMap<>();
map.computeIfAbsent("key", k -> new ArrayList<>()).add("value");
// computeIfAbsent is atomic – safe from race; simple get+put is NOT

// 3. Frequency map (classic interview pattern)
Map<String, Long> freq = words.stream()
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));

// 4. Multi-field sort
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getDepartment)
        .thenComparing(Employee::getSalary, Comparator.reverseOrder()))
    .collect(Collectors.toList());

// 5. Safe iteration with removal
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().startsWith("FAIL")) {
```

```

        it.remove(); // safe – not list.remove()
    }
}

```

COMPARISON TABLES

Operation	ArrayList	LinkedList	ArrayDeque
add(end)	O(1) amort	O(1)	O(1) amort
add(front)	O(n)	O(1)	O(1) amort
get(index)	O(1)	O(n)	O(n)
remove(middle)	O(n)	O(n)*	O(n)
Memory	compact	node overhead	compact

*O(1) if you have the `ListIterator` positioned there.

Map	Ordered?	Null Key?	Thread-Safe?	Perf
HashMap	No	1 null	✗	O(1) avg
LinkedHashMap	Insertion/Access	1 null	✗	O(1) avg
TreeMap	Sorted	✗	✗	O(log n)
ConcurrentHashMap	No	✗	✓	O(1) avg
Hashtable	No	✗	✓ (coarse)	O(1) avg

INTERVIEW TRAP ANSWERS

- **"What is `ConcurrentModificationException` ?"** — Thrown when a collection's structural modification count (`modCount`) changes during iteration via a fail-fast iterator. It is NOT guaranteed to throw — it's best-effort. Cannot be relied upon for correctness.
- **"Why is `HashMap` O(n) worst case?"** — All keys hash to same bucket → single linked list → O(n) traversal. Java 8 mitigates with treeification (red-black tree) at 8 nodes → O(log n) worst case.
- **"`HashSet` vs `TreeSet` — when to use which?"** — `HashSet` for O(1) add/contains/remove with no order requirement. `TreeSet` when you need sorted iteration or range queries (`headSet` , `subSet` , `tailSet`).
- **"How does `PriorityQueue` order elements?"** — Min-heap: smallest element polled first. For max-heap: `new PriorityQueue<>(Comparator.reverseOrder())` . The heap invariant is maintained during offer/poll, not throughout internal array.

ARCHITECTURE BOUNDARY

```

java.util Collections Hierarchy
├─ Iterable → Collection
│   ├── List: ArrayList, LinkedList, CopyOnWriteArrayList
│   ├── Set: HashSet, LinkedHashSet, TreeSet, CopyOnWriteArraySet
│   └─ Queue/Deque: PriorityQueue, ArrayDeque, LinkedBlockingQueue
├─ Map (separate hierarchy)
│   ├── HashMap, LinkedHashMap, TreeMap
│   ├── ConcurrentHashMap (java.util.concurrent)
│   └─ EnumMap, WeakHashMap, IdentityHashMap
└─ Concurrent Package (java.util.concurrent)
    ├── BlockingQueue: ArrayBlockingQueue, LinkedBlockingQueue
    ├── ConcurrentMap: ConcurrentHashMap, ConcurrentSkipListMap
    └─ CopyOnWrite: CopyOnWriteArrayList, CopyOnWriteArraySet

```

37.3 One-Page Selenium Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
WebDriver protocol	W3C standard JSON-over-HTTP protocol; client (your code) ↔ driver (chromedriver) ↔ browser	Old Selenium RC/RC2 injected JS; W3C WebDriver is native browser automation
<code>findElement</code> vs <code>findElements</code>	<code>findElement</code> throws <code>NoSuchElementException</code> if not found; <code>findElements</code> returns empty list	Never use <code>findElements().get(0)</code> without size check — throws <code>IndexOutOfBoundsException</code>
Explicit Wait	<code>WebDriverWait</code> + <code>ExpectedConditions</code> — polls every 500ms up to timeout	Does NOT use <code>Thread.sleep()</code> ; uses polling. Never mix with implicit wait — they interact unpredictably
Implicit Wait	Global setting; waits N seconds for element to appear on every <code>findElement</code> call	Applies globally and stacks with explicit wait — set implicit to 0 when using explicit
Fluent Wait	Custom polling interval + exception ignore list — most flexible	<code>ExpectedConditions</code> class has 30+ built-in conditions; write custom <code>Function<WebDriver, T></code> for complex cases
Stale Element	<code>StaleElementReferenceException</code> — element DOM detached after page refresh/navigation	Re-locate element inside retry loop; never store element refs across page transitions
<code>JavascriptExecutor</code>	Executes JS in browser context; bypasses Selenium interactability checks	Clicking via JS bypasses real user events — use only for scrolling, badges, tricky elements
<code>Actions</code> class	Chains user gestures: hover, drag-drop, key sequences	<code>Actions</code> builder must call <code>.perform()</code> to execute; not all browsers support all actions equally
PageFactory / <code>@FindBy</code>	DI-based element locating with lazy initialization; uses <code>PageFactory.initElements()</code>	<code>@FindBy</code> elements are re-looked up on each access (proxied) — not cached; can still go stale
<code>WebDriverManager</code>	Automatically downloads/configures correct driver binary	In CI, use <code>--no-sandbox --disable-dev-shm-usage</code> Chrome flags; headless mode with <code>--headless=new</code>
Frame handling	Must <code>switchTo().frame()</code> before interacting with iframe content	Must <code>switchTo().defaultContent()</code> to return; nested frames require sequential switches
Window handles	<code>getWindowHandles()</code> returns <code>Set<String></code> (no guaranteed order)	Iterate set and switch to the handle that isn't current to get newly opened window
<code>Select</code> class	Wrapper for <code><select></code> dropdowns: <code>selectByValue</code> , <code>selectByVisibleText</code>	Does NOT work for custom JS dropdowns (div-based) — use <code>Actions</code> or click/find pattern
Alert	<code>switchTo().alert()</code> for browser dialogs	Must handle alert before interacting with page; <code>NoAlertPresentException</code> if none exists
XPath axes	<code>ancestor</code> , <code>following-sibling</code> , <code>preceding-sibling</code> , <code>descendant</code> for relational locators	<code>//div[@class='x']</code> fails if class has multiple values — use <code>contains(@class, 'x')</code>

CRITICAL CODE PATTERNS

```
// 1. Robust explicit wait (set implicit to 0 first)
driver.manage().timeouts().implicitlyWait(Duration.ZERO);
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
WebElement element = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("submit"))
);

// 2. Custom fluent wait with ignored exceptions
Wait<WebDriver> fluentWait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofMillis(500))
    .ignoring(NoSuchElementException.class)
    .ignoring(StaleElementReferenceException.class);

WebElement result = fluentWait.until(d ->
    d.findElement(By.cssSelector(".result:not(.loading)"))
);

// 3. Stale element retry
public WebElement findWithRetry(By locator, int maxRetries) {
```

```

    for (int i = 0; i < maxRetries; i++) {
        try {
            return driver.findElement(locator);
        } catch (StaleElementReferenceException e) {
            if (i == maxRetries - 1) throw e;
        }
    }
    throw new NoSuchElementException("Element not found after retries: " + locator);
}

// 4. Scroll into view and click
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].scrollIntoView({block:'center'})", element);
element.click();

// 5. Window handle switching
String parentWindow = driver.getWindowHandle();
Set<String> allWindows = driver.getWindowHandles();
for (String handle : allWindows) {
    if (!handle.equals(parentWindow)) {
        driver.switchTo().window(handle);
        break;
    }
}

// 6. Page Object Model structure
public class LoginPage {
    private final WebDriver driver;
    private final WebDriverWait wait;

    @FindBy(id = "username") private WebElement usernameField;
    @FindBy(id = "password") private WebElement passwordField;
    @FindBy(css = "button[type='submit']") private WebElement loginButton;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        PageFactory.initElements(driver, this);
    }

    public DashboardPage login(String user, String pass) {
        wait.until(ExpectedConditions.visibilityOf(usernameField)).sendKeys(user);
        passwordField.sendKeys(pass);
        loginButton.click();
        return new DashboardPage(driver);
    }
}

```

COMPARISON TABLES

Wait Type	Scope	Polling	Exception Ignore	Best For
Implicit	Global	browser native	NoSuchElement	Simple scripts only
Explicit	Per-call	500ms default	configurable	Production tests
Fluent	Per-call	custom	custom list	Complex dynamic UIs
<code>Thread.sleep()</code>	Global pause	N/A	N/A	Never in production

Locator	Reliability	Speed	Use When
<code>By.id</code>	★★★★★	Fastest	ID available
<code>By.cssSelector</code>	★★★★★	Fast	Modern apps
<code>By.xpath</code>	★★★★	Slower	Complex traversal needed
<code>By.name</code>	★★★★	Fast	Form fields

Locator	Reliability	Speed	Use When
By.linkText	★★	Fast	Exact link text
By.className	★★	Fast	Unique class only

INTERVIEW TRAP ANSWERS

- **"Implicit + Explicit wait interaction?"** — Explicit wait uses `WebDriverWait.until()` with its own polling. If implicit wait is set, it adds its full timeout ON TOP of the explicit wait's polling — effective timeout can be implicit + explicit. Always set implicit to 0 when using explicit waits.
- **"Why does PageFactory still get stale elements?"** — `@FindBy` elements are proxies that re-call `findElement()` on each access. But between the access and the action (e.g., `.click()`), the DOM can change, causing staleness. The proxy re-locates but doesn't prevent the race.
- **"How do you test file downloads in Selenium?"** — Configure Chrome download directory via `ChromeOptions`, then poll the filesystem for the file's appearance. Never test downloads through the browser dialog (bypass it via prefs).
- **"Difference between driver.close() and driver.quit()?"** — `close()` closes the current window (one window handle). `quit()` destroys the entire WebDriver session, kills the browser process. Always call `quit()` in `@AfterClass` / `@After`.

ARCHITECTURE BOUNDARY

```

Test Code (Java/Python)
  | HTTP/JSON (W3C WebDriver Protocol)
  ▼
WebDriver Client (selenium-java.jar)
  | localhost:PORT HTTP
  ▼
Browser Driver (chromedriver / geckodriver / msedgedriver)
  | DevTools Protocol / internal IPC
  ▼
Browser (Chrome / Firefox / Edge)
  |
  ▼
Rendered Page (DOM, JS engine, network)

```

37.4 One-Page TestNG Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
<code>@Test</code>	Marks method as test; supports <code>priority</code> , <code>groups</code> , <code>dependsOnMethods</code> , <code>enabled</code> , <code>dataProvider</code>	<code>priority</code> is not absolute order — lower number runs first, but group ordering can override
<code>@BeforeSuite</code> / <code>@AfterSuite</code>	Runs once per entire test suite (XML <code><suite></code>)	Runs in the suite's config thread — not the test's thread; thread-safety matters in parallel mode
<code>@BeforeClass</code> / <code>@AfterClass</code>	Runs once per test class (static equivalent in JUnit)	NOT <code>static</code> in TestNG (unlike JUnit's <code>@BeforeClass</code>) — runs in the first test method's thread
<code>@BeforeMethod</code> / <code>@AfterMethod</code>	Runs before/after each <code>@Test</code> method	Can receive <code>Method</code> / <code>ITestResult</code> parameters for test-aware setup/teardown
<code>@DataProvider</code>	Supplies test data as <code>Object[][]</code> or <code>Iterator<Object[]></code>	Return type must be <code>Object[][]</code> or <code>Iterator<Object[]></code> — <code>List</code> won't work; annotate with <code>parallel=true</code> for data parallelism
<code>@Factory</code>	Creates multiple test instances with different configs at runtime	Different from <code>@DataProvider</code> — <code>@Factory</code> creates different class instances; <code>@DataProvider</code> calls same instance multiple times
<code>dependsOnMethods</code>	Test skipped (not failed) if dependency fails	Soft dependency: <code>alwaysRun=true</code> runs test even if dependency failed — distinguish skip vs fail in reports

Concept	One-Line Definition	Critical Trap
groups	Logical test grouping; run via XML <code><groups><run><include></code>	<code>@Test(groups={"smoke", "regression"})</code> — method belongs to multiple groups simultaneously
<code>ITestListener</code>	Event listener: <code>onTestStart</code> , <code>onTestSuccess</code> , <code>onTestFailure</code> , <code>onTestSkipped</code>	Implement <code>ITestListener</code> , register in XML <code><listeners></code> or with <code>@Listeners</code> annotation
Parallel execution	<code>parallel="methods"/"classes"/"tests"/"instances"</code> in <code>testng.xml</code>	<code>parallel="methods"</code> requires thread-safe test design; shared <code>WebDriver</code> = race condition
<code>@Listeners</code>	Class-level annotation to attach listeners	<code>@Listeners({ScreenshotListener.class, ReportListener.class})</code> — multiple listeners supported
SoftAssert	Collects all assertion failures; reports at <code>assertAll()</code>	<code>assertAll()</code> MUST be called at end; forgetting it silently swallows all failures
<code>Assert.assertEquals(actual, expected)</code>	Compares actual to expected	Order matters for error messages: <code>assertEquals(actual, expected)</code> — message says "expected X but got Y"
<code>IRetryAnalyzer</code>	Interface for automatic test retry on failure	Set max retry count; mark test with <code>@Test(retryAnalyzer=RetryAnalyzer.class)</code>
<code>testng.xml</code>	Suite configuration: classes, methods, groups, parallel settings, listeners	<code><parameter></code> tag passes values to <code>@Parameters</code> — string only; type conversion manual

CRITICAL CODE PATTERNS

```
// 1. DataProvider with parallel execution
@DataProvider(name = "loginData", parallel = true)
public Object[][] loginData() {
    return new Object[][] {
        {"admin@test.com", "Admin@123", "ADMIN"},
        {"user@test.com", "User@123", "USER"},
        {"guest@test.com", "Guest@123", "GUEST"}
    };
}

@Test(dataProvider = "loginData", groups = {"smoke", "regression"})
public void testLogin(String email, String password, String expectedRole) {
    // test body
}

// 2. Thread-safe WebDriver management
public class BaseTest {
    protected static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    @BeforeMethod(alwaysRun = true)
    public void setUp(Method method) {
        ChromeOptions opts = new ChromeOptions();
        opts.addArguments("--headless=new", "--no-sandbox");
        driver.set(new ChromeDriver(opts));
        driver.get().manage().window().maximize();
        System.out.println("Starting: " + method.getName());
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) {
            captureScreenshot(result.getName());
        }
        if (driver.get() != null) {
            driver.get().quit();
            driver.remove(); // prevent memory leak
        }
    }
}
```

```
// 3. SoftAssert usage
@Test
public void validateDashboard() {
    SoftAssert soft = new SoftAssert();
    soft.assertEquals(dashboard.getTitle(), "Dashboard", "Title mismatch");
    soft.assertTrue(dashboard.isLogoutVisible(), "Logout not visible");
    soft.assertNotNull(dashboard.getUserProfile(), "Profile is null");
    soft.assertAll(); // MUST call – reports all failures together
}

// 4. IRetryAnalyzer
public class RetryAnalyzer implements IRetryAnalyzer {
    private int retryCount = 0;
    private static final int MAX_RETRY = 2;

    @Override
    public boolean retry(ITestResult result) {
        if (retryCount < MAX_RETRY) {
            retryCount++;
            return true; // retry
        }
        return false; // give up
    }
}

// 5. Custom Listener for screenshots
public class ScreenshotListener implements ITestListener {
    @Override
    public void onTestFailure(ITestResult result) {
        WebDriver driver = ((BaseTest) result.getInstance()).driver.get();
        File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        String path = "reports/screenshots/" + result.getName() + "_" +
            System.currentTimeMillis() + ".png";
        FileUtils.copyFile(screenshot, new File(path));
    }
}
```

COMPARISON TABLES

Annotation	JUnit 5 Equivalent	Scope
@BeforeSuite	@BeforeAll (on suite level)	Entire suite
@BeforeClass	@BeforeAll (static)	Class
@BeforeMethod	@BeforeEach	Each test method
@Test(priority)	@Order	Method ordering
@DataProvider	@MethodSource / @CsvSource	Parameterization
@Factory	no direct equivalent	Dynamic instances

Parallel Mode	Scope	Thread Per
methods	All @Test methods	Each method
classes	Classes in <test>	Each class
tests	<test> tags in XML	Each <test> tag
instances	Class instances	Each object instance

INTERVIEW TRAP ANSWERS

- **"Difference between @Factory and @DataProvider ?"** — @DataProvider calls the SAME test class instance with different parameter sets. @Factory creates DIFFERENT instances of the test class for each set — useful when constructor parameters vary (e.g., different browsers).

- "What happens when a `dependsOnMethods` test fails?" — Dependent test is SKIPPED (not failed). It appears as `SKIP` in report. If `alwaysRun=true`, dependent test still runs regardless of dependency result.
- "Why is `SoftAssert` not thread-safe?" — `SoftAssert` instance collects failures in an internal list. If shared across threads (e.g., in `@DataProvider(parallel=true)` test), assertions from different threads corrupt each other. Create one per test method.
- "How does TestNG determine test execution order?" — Priority (lower first), then alphabetical within same priority. Groups and `dependsOnMethods` can override. XML `preserve-order="true"` preserves class declaration order.

ARCHITECTURE BOUNDARY

```
testng.xml
├─ <suite parallel="classes" thread-count="4">
│   └─ <listeners> → ITestListener, ISuiteListener, IReporter
│       └─ <test>
│           └─ <groups><run><include name="smoke"/>
│               └─ <classes>
│                   └─ <class name="com.tests.LoginTest">
│                       └─ <methods><include name="testLogin"/>
│                           └─ @DataProvider → injects params per invocation
```

Execution Flow:

```
@BeforeSuite → @BeforeTest → @BeforeClass → @BeforeMethod
→ @Test → @AfterMethod → @AfterClass → @AfterTest → @AfterSuite
```

37.5 One-Page REST Assured Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
<code>given()</code> / <code>when()</code> / <code>then()</code>	BDD-style DSL: setup → action → assertion	<code>given()</code> is optional (static import); <code>when()</code> executes request; <code>then()</code> starts assertion chain
<code>RequestSpecification</code>	Reusable request config: base URI, headers, auth, content-type	Extract via <code>given().spec(spec)</code> — reduces duplication across test class
<code>ResponseSpecification</code>	Reusable response assertions: status, content-type, body fields	Share via <code>RestAssured.responseSpecification</code> global default
<code>ExtractableResponse</code>	Tap out of assertion chain to extract values for later use	<code>.extract().path("data.id")</code> returns <code>Object</code> ; <code>.extract().as(MyClass.class)</code> deserializes
<code>JSONPath</code>	GPath-based path for JSON extraction: <code>data[0].name</code> , <code>find {it.age > 18}</code>	Groovy-style GPath, not JavaScript path — <code>\$</code> not needed; list filter uses Groovy closure syntax
<code>body(matchesJsonSchema(...))</code>	Schema validation against JSON Schema file	Requires <code>rest-assured-json-schema</code> dependency; schema must be on classpath
Serialization	<code>body(myObject)</code> serializes POJO if Jackson/Gson on classpath	Set <code>ContentType.JSON</code> explicitly; without it, may send as form or plain text
Auth methods	<code>.auth().basic()</code> , <code>.oauth2()</code> , <code>.preemptive().basic()</code>	<code>preemptive().basic()</code> sends credentials without waiting for 401 challenge — use for non-RFC-compliant servers
<code>filter()</code>	Hook into request/response lifecycle: logging, custom auth, metrics	<code>new RequestLoggingFilter()</code> / <code>new ResponseLoggingFilter()</code> for debug; <code>new AllureRestAssured()</code> for reports
<code>baseURI</code> / <code>basePath</code>	Global config: <code>RestAssured.baseURI</code> , <code>RestAssured.basePath</code>	Thread-safety issue — global state shared across threads; use <code>RequestSpecification</code> instead in parallel tests
Relaxed HTTPS	<code>RestAssured.useRelaxedHTTPSValidation()</code>	Disables SSL cert validation — never in production; only for self-signed cert test environments

Concept	One-Line Definition	Critical Trap
Form params	<code>.formParam("key", "value")</code> for <code>application/x-www-form-urlencoded</code>	Do NOT use <code>.body()</code> for form submissions — use <code>.formParam()</code> and <code>ContentType.URLENC</code>
Multipart	<code>.multiPart("file", new File("test.pdf"))</code>	Content type auto-set to <code>multipart/form-data</code> ; check server boundary handling
JsonPath vs XmlPath	<code>from(responseBody).get("path")</code> for static parsing outside REST Assured chain	<code>JsonPath jp = JsonPath.from(json); jp.getList("items.name")</code> extracts list directly
Cookie handling	<code>.cookie("name", "value")</code> or <code>.cookies(map)</code>	<code>.then().cookie("JSESSIONID")</code> asserts cookie presence; <code>.extract().cookie("name")</code> gets value

CRITICAL CODE PATTERNS

```
// 1. Reusable RequestSpec (production pattern)
public class ApiConfig {
    public static RequestSpecification requestSpec() {
        return new RequestSpecBuilder()
            .setBaseUrl("https://api.staging.example.com")
            .setBasePath("/v2")
            .setContentType(ContentType.JSON)
            .addHeader("X-API-Key", System.getenv("API_KEY"))
            .setRelaxedHTTPSValidation()
            .addFilter(new AllureRestAssured())
            .addFilter(RequestLoggingFilter.logRequestTo(System.out))
            .build();
    }
}

// 2. Full CRUD test
@Test
public void testCreateAndRetrieveUser() {
    Map<String, Object> payload = Map.of(
        "name", "John Doe",
        "email", "john@example.com",
        "role", "TESTER"
    );

    // CREATE
    String userId = given()
        .spec(ApiConfig.requestSpec())
        .body(payload)
        .when()
        .post("/users")
        .then()
        .statusCode(201)
        .body("data.name", equalTo("John Doe"))
        .body("data.role", equalTo("TESTER"))
        .extract().path("data.id");

    // READ
    given()
        .spec(ApiConfig.requestSpec())
        .when()
        .get("/users/{id}", userId)
        .then()
        .statusCode(200)
        .body("data.id", equalTo(userId));
}

// 3. JSON Schema validation
given()
    .spec(ApiConfig.requestSpec())
    .when()
    .get("/products")
    .then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath("schemas/products-schema.json"));

// 4. Extract list and assert
```

```

List<String> names = given()
    .spec(ApiConfig.requestSpec())
    .queryParams("status", "ACTIVE")
    .when()
    .get("/users")
    .then()
    .statusCode(200)
    .extract().jsonPath().getList("data.name", String.class);

assertThat(names).hasSize(5).contains("Admin User");

// 5. OAuth2 + retry on 429
given()
    .spec(ApiConfig.requestSpec())
    .auth().oauth2(tokenProvider.getToken())
    .queryParams("page", 1)
    .when()
    .get("/reports")
    .then()
    .statusCode(anyOf(equalTo(200), equalTo(202))));

```

COMPARISON TABLES

Auth Method	When to Use	Sends Credentials
<code>.basic(u, p)</code>	Standard HTTP Basic Auth	After 401 challenge
<code>.preemptive().basic(u, p)</code>	Non-RFC-compliant servers	Every request, no challenge
<code>.oauth2(token)</code>	Bearer token auth	<code>Authorization: Bearer <token></code> header
<code>.oauth(...)</code>	OAuth 1.0a	Signed headers (4-param)
<code>.form(u, p, config)</code>	Form-based auth flow	POST to login endpoint

Extraction Method	Returns	Use For
<code>.path("a.b.c")</code>	Object	Single value
<code>.jsonPath().getList("items")</code>	List	Array fields
<code>.as(MyClass.class)</code>	Deserialized POJO	Full body mapping
<code>.asString()</code>	raw JSON string	Custom parsing
<code>.header("X-Header")</code>	String	Response headers
<code>.cookie("name")</code>	String	Session cookies

INTERVIEW TRAP ANSWERS

- **"REST Assured is not thread-safe with global config?"** — `RestAssured.baseURI`, `RestAssured.requestSpecification` etc. are static fields — shared across all threads. In parallel tests, use `RequestSpecBuilder` to create thread-local specs per test.
- **"Difference between `body("field", equalTo())` and `body("field", is())`?"** — `is()` is Hamcrest shorthand for `equalTo()`. Functionally identical. Both are Hamcrest matchers. `equalTo` is more explicit.
- **"How do you test paginated APIs?"** — Loop while `response.path("meta.hasNext") == true`, extract `nextPageToken`, use as query param in next request. Collect all results across pages and assert aggregate.
- **"How do you validate response time?"** — `.then().time(lessThan(2000L))` (milliseconds). Or `.extract().time(TimeUnit.MILLISECONDS)` for manual assertion.

ARCHITECTURE BOUNDARY

```

REST Assured DSL
├─ given() → RequestSpecification (URI, headers, auth, body, params)
├─ when() → HTTP Execution (GET/POST/PUT/PATCH/DELETE)
└─ then() → ValidatableResponse (status, headers, body matchers)
    └─ extract() → ExtractableResponse (values for reuse)

```

Underlying Stack:
 REST Assured → Apache HttpClient → HTTP → API Server
 → Jackson/Gson (serialization)
 → GPath/JsonPath (assertion)
 → Hamcrest (matchers)
 → JSON Schema Validator

37.6 One-Page Playwright Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
Auto-waiting	Playwright auto-waits for actionability: visible, enabled, stable, not covered	Default 30s timeout per action; customize via <code>page.setDefaultTimeout()</code> or per-action <code>timeout</code> option
Locator API	Chainable element references; re-queried on each action	<code>page.locator()</code> vs <code>page.\$()</code> — <code>locator()</code> is recommended; <code>\$()</code> returns <code>ElementHandle</code> (old API)
<code>expect()</code>	Async assertion with built-in retry; replaces manual waits	<code>await expect(locator).toBeVisible()</code> retries automatically; direct <code>locator.isVisible()</code> is snapshot (no retry)
Playwright context	Browser → BrowserContext → Page hierarchy; context isolates cookies/storage	Each <code>BrowserContext</code> is isolated like a separate browser profile; share driver across tests via context
<code>page.route()</code>	Intercept and mock network requests	<code>route.fulfill()</code> mocks response; <code>route.continue()</code> passes through; <code>route.abort()</code> simulates failure
<code>APIRequestContext</code>	Built-in HTTP client sharing auth/cookies with browser context	<code>playwright.request.newContext()</code> for standalone API calls; shares cookies with <code>page.context()</code> if from same context
<code>page.evaluate()</code>	Execute JS in page context; returns serializable result	Cannot pass non-serializable DOM objects; use <code>evaluateHandle()</code> for DOM object references
Fixtures	Playwright Test's DI system: <code>test.extend()</code> creates typed fixtures	Fixtures can have <code>scope: 'test'</code> (default) or <code>scope: 'worker'</code> for shared setup
<code>page.waitForResponse()</code>	Waits for matching network response (URL/predicate)	Must start waiting BEFORE triggering the action — race condition if action triggers immediately
Trace Viewer	Records DOM snapshots, actions, console, network for every step	Enable via <code>trace: 'on-first-retry'</code> in <code>playwright.config.ts</code> — view with <code>npx playwright show-trace</code>
<code>storageState</code>	Saves cookies + localStorage to JSON for auth reuse	Set in <code>playwright.config.ts</code> under <code>use.storageState</code> to skip login per test
<code>page.addInitScript()</code>	Runs script in page before any JavaScript loads	Ideal for stubbing globals, disabling analytics, setting feature flags before page JS initializes
Shadow DOM	<code>locator.locator()</code> pierces open shadow roots automatically	Closed shadow roots cannot be pierced — requires <code>page.evaluate()</code> with <code>shadowRoot</code> access
Mobile emulation	<code>devices['iPhone 13']</code> preset includes viewport, UA, touch	<code>hasTouch: true</code> required for touch events; emulation is viewport-level, not actual mobile rendering
<code>test.use()</code>	Override <code>playwright.config.ts</code> defaults per describe block	<code>test.use({ viewport: {width:1920,height:1080} })</code> — scoped to that describe block only

CRITICAL CODE PATTERNS

```
// 1. Playwright config (playwright.config.ts)
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
```

```

retries: process.env.CI ? 2 : 0,
workers: process.env.CI ? 4 : undefined,
reporter: [['html'], ['allure-playwright']],
use: {
  baseURL: process.env.BASE_URL ?? 'https://staging.example.com',
  trace: 'on-first-retry',
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
},
projects: [
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  { name: 'Mobile Chrome', use: { ...devices['Pixel 7'] } },
],
});

// 2. Custom fixture (typed DI)
import { test as base, Page } from '@playwright/test';
import { LoginPage } from './pages/LoginPage';

type MyFixtures = {
  loginPage: LoginPage;
  authenticatedPage: Page;
};

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await use(loginPage);
  },
  authenticatedPage: async ({ page }, use) => {
    // Reuse stored session
    await page.context().addCookies(loadSavedCookies());
    await use(page);
  },
});

// 3. Network interception / mocking
await page.route('**/api/products**', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ data: mockProducts }),
  });
});

// 4. Wait for response BEFORE action (race-condition-safe)
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/login') && r.status() === 200),
  page.locator('#submit').click(), // triggers request
]);
const responseBody = await response.json();
expect(responseBody.token).toBeTruthy();

// 5. Page Object Model (TypeScript)
export class LoginPage {
  readonly usernameInput = this.page.locator('[data-testid="username"]');
  readonly passwordInput = this.page.locator('[data-testid="password"]');
  readonly submitButton = this.page.locator('button[type="submit"]');
  readonly errorMessage = this.page.locator('.error-banner');

  constructor(private readonly page: Page) {}

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
    await expect(this.page).toHaveURL(/dashboard/);
  }

  async expectError(message: string) {
    await expect(this.errorMessage).toHaveText(message);
  }
}

```

```
// 6. API testing with shared auth context
const apiContext = await playwright.request.newContext({
  baseURL: 'https://api.example.com',
  extraHTTPHeaders: { 'Authorization': `Bearer ${token}` },
});
const response = await apiContext.get('/users');
expect(response.ok()).toBeTruthy();
```

COMPARISON TABLES

Feature	Playwright	Selenium
Auto-wait	✅ Built-in	❌ Manual waits
Network mocking	✅ Native	⚠️ Proxy required
Multi-browser	Chromium, Firefox, WebKit	All via drivers
Parallel	Workers (process)	Thread-based
Mobile emulation	✅ Built-in	⚠️ Limited
Trace/Video	✅ Built-in	❌ 3rd party
Protocol	CDP + WebKit protocol	W3C WebDriver
API testing	✅ Built-in	❌ Separate tool

Locator Strategy	Selector	Priority
<code>getByRole</code>	ARIA role + name	1st (accessibility)
<code>getByTestId</code>	<code>data-testid</code>	2nd (stable)
<code>getByLabel</code>	form label	3rd (semantic)
<code>getByText</code>	visible text	4th
<code>locator(css)</code>	CSS selector	5th
<code>locator(xpath)</code>	XPath	Last resort

INTERVIEW TRAP ANSWERS

- **"How does Playwright auto-waiting work?"** — Before each action, Playwright checks actionability conditions: element exists in DOM, is visible, is stable (not animating), is enabled, receives pointer events (not covered). It polls these conditions up to the timeout.
- **"Difference between `locator.isVisible()` and `expect(locator).toBeVisible()`?"** — `isVisible()` is a point-in-time snapshot; returns immediately (no retry). `expect().toBeVisible()` retries until visible or timeout. Always use `expect()` in assertions.
- **"How do you handle flaky tests in Playwright?"** — Enable `retries: 2` in config. Use `trace: 'on-first-retry'` to record what happened on failure. Use `storageState` to avoid auth flakiness. Use `data-testid` locators for stability. Analyze traces in Playwright UI.
- **"Workers vs threads in Playwright?"** — Playwright Test uses worker processes (not threads) for parallelism. Each worker runs tests in its own Node.js process, completely isolated. `fullyParallel: true` means each test file gets its own worker.

ARCHITECTURE BOUNDARY

```
playwright.config.ts
├─ projects[] → Browser × Viewport × Device combinations
├─ workers → N parallel worker processes
└─ use{} → Global defaults (baseURL, trace, screenshot)

Per Worker Process:
Browser (Chromium/Firefox/WebKit)
├─ BrowserContext (isolated session, cookies, storage)
│   └─ Page (tab)
│       ├── Actions → Auto-wait → CDP/WebKit protocol → Browser
│       └─ page.route() → Network interception layer
```

```

└─ page.evaluate() → JS execution in page context
└─ expect() → Async assertion with retry

```

37.7 One-Page CI/CD Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
Pipeline	Automated sequence of stages from code commit to production delivery	Stages are logically sequential; jobs within a stage can run in parallel
Fail-fast	Abort pipeline immediately on first failure to save compute and give fast feedback	Balance with: don't fail-fast unit tests that share build artifacts needed for later stages
Artifact	Build output passed between stages (JAR, Docker image, test reports)	Artifacts have TTL (expiry); always pin artifact names to commit SHA for traceability
DORA Metrics	Deployment Frequency, Lead Time for Changes, MTTR, Change Failure Rate	These are team-level metrics; use them to justify investment in test automation ROI
Blue-Green	Two identical environments; traffic switch via load balancer; instant rollback	Requires 2x infrastructure cost; DB migrations must be backward-compatible during switch
Canary	Gradual traffic shift (1% → 10% → 50% → 100%) with automated health gates	Rollback only affects canary users; requires feature flags and real-time monitoring
Rolling	Replace instances one-by-one; no extra infrastructure cost	Can't instant rollback; during rollout, old and new versions coexist → must be API-compatible
Feature Flags	Decouple deployment from release; enable/disable features at runtime	Flag debt accumulates fast; define lifecycle (creation date, owner, cleanup date)
Self-hosted runner	Your own machine/VM/container runs CI jobs	Security risk: untrusted PRs can run code on your infrastructure; use ephemeral runners
Cache	Speed up builds by reusing dependencies between runs	Cache key strategy matters: <code>hashFiles('**/pom.xml')</code> invalidates only on dep change
Matrix strategy	Run same job with multiple parameter combinations (OS × Java version × browser)	Matrix explodes: 3 OS × 3 Java × 3 browsers = 27 jobs; restrict with <code>include / exclude</code>
Environment secrets	Encrypted key-value pairs injected at runtime; not visible in logs	Never interpolate secrets into shell variables that echo to stdout; use <code>--secret</code> pattern
Staged gates	Manual approval between stages (UAT → Prod)	Automate gate criteria where possible: test coverage%, performance baseline, security scan
Ephemeral environments	Spin up complete stack per PR; destroy after merge	Requires containerized app + infrastructure-as-code; namespace isolation in Kubernetes
SAST/DAST	Static analysis pre-build; Dynamic analysis against running app	SAST: SonarQube, SpotBugs, Semgrep. DAST: OWASP ZAP, Burp. Both in pipeline, never in prod

7-STAGE PIPELINE MODEL

Stage 1: SOURCE • Trigger: push/PR • Checkout code • Lint / format check • Secrets scan	Stage 2: BUILD • Compile • Dependency resolution • Package (JAR/WAR/Image) • Version tagging • Push to registry	Stage 3: TEST (Unit) • Unit tests (JUnit/TestNG) • Code coverage gate (≥80%) • SAST scan (SonarQube) • Mutation testing (optional) • Fail-fast: YES
Stage 4: TEST (Integ)	Stage 5: TEST (E2E)	Stage 6: STAGING/UAT

- Integration tests
- API contract tests
- DB migration tests
- Component tests
- Fail-fast: YES
- Selenium/Playwright
- Smoke suite (fast)
- Regression suite
- Cross-browser matrix
- Screenshot on failure
- Deploy to staging env
- Performance tests (JMeter)
- DAST scan (OWASP ZAP)
- Manual exploratory testing
- Approval gate

Stage 7: PRODUCTION

- Deployment strategy: Canary (1%→10%→50%→100%)
- Smoke tests against prod (critical paths only)
- Synthetic monitoring / health checks
- Automatic rollback if error rate > threshold
- DORA metrics recorded: deploy frequency, lead time

GHA vs JENKINS vs GITLAB SYNTAX

```
# — GitHub Actions —————
name: CI Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        java: [17, 21]
        browser: [chrome, firefox]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: ${ matrix.java }
          cache: maven
      - name: Run tests
        run: mvn test -Dbrowser=${ matrix.browser }
        env:
          API_KEY: ${ secrets.API_KEY }
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-results-${ matrix.java }-${ matrix.browser }
          path: target/surefire-reports/

# — Jenkins (Declarative Pipeline) —————
pipeline {
  agent { docker { image 'maven:3.9-eclipse-temurin-21' } }
  environment {
    API_KEY = credentials('api-key-secret')
  }
  stages {
    stage('Build') {
      steps { sh 'mvn compile' }
    }
    stage('Test') {
      parallel {
        stage('Unit') { steps { sh 'mvn test -P unit' } }
        stage('API') { steps { sh 'mvn test -P api' } }
      }
    }
    stage('E2E') {
      when { branch 'main' }
      steps { sh 'mvn test -P e2e -Dbrowser=chrome' }
    }
  }
  post {
    always {
      junit 'target/surefire-reports/*.xml'
      publishHTML(target: [reportDir: 'target/site/allure', reportName: 'Allure'])
    }
  }
}
```

```

    failure { slackSend(message: "Build FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}") }
  }
}

# — GitLab CI —————
stages: [build, test, e2e, deploy]

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"

build:
  stage: build
  cache:
    key: { files: [pom.xml] }
    paths: [.m2/repository]
  script: mvn package -DskipTests
  artifacts:
    paths: [target/*.jar]

e2e-chrome:
  stage: e2e
  image: mcr.microsoft.com/playwright:v1.44.0-jammy
  script:
    - npx playwright test --project=chromium
  artifacts:
    when: always
    paths: [playwright-report/]
    expire_in: 7 days
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

```

DEPLOYMENT STRATEGIES

Strategy	Traffic Shift	Rollback Speed	Infra Cost	DB Compat Required
Recreate	All at once (downtime)	Redeploy old	Low	No
Rolling	Instance by instance	Slow (re-roll)	None extra	YES
Blue-Green	Instant (LB flip)	Instant (LB flip)	2x	YES
Canary	Gradual (1% → 100%)	Instant (reroute)	~10% extra	YES
Shadow	No real traffic	N/A (observe)	2x	No

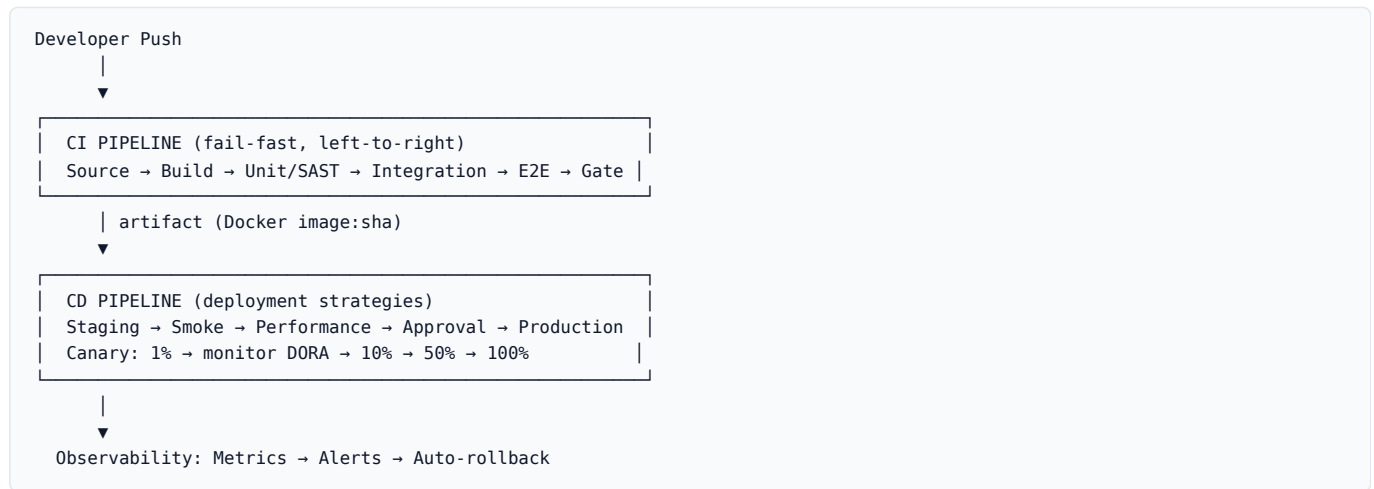
DORA METRICS

Metric	Elite	High	Medium	Low
Deployment Frequency	Multiple/day	Weekly	Monthly	< Monthly
Lead Time for Changes	< 1 hour	< 1 day	< 1 week	> 6 months
MTTR	< 1 hour	< 1 day	< 1 week	> 1 week
Change Failure Rate	0-5%	5-10%	10-15%	> 15%

INTERVIEW TRAP ANSWERS

- **"Why put unit tests before integration tests in pipeline?"** — Fail-fast principle: unit tests are fast (seconds) and cheap. Failing fast avoids wasting compute on 20-min integration/E2E suites. Signal speed is the priority.
- **"How do you prevent flaky tests from blocking CI?"** — Quarantine zone: flaky tests auto-tagged and moved to non-blocking suite. Alert sent. Fix within SLA (e.g., 3 days) or delete. Use retry (max 2) with jitter for transient failures.
- **"Blue-green with database schema changes?"** — DB changes must be backward-compatible: add columns with defaults, never drop/rename immediately. Deploy DB migration first, then blue-green app switch. Remove old schema in next release (expand-contract pattern).
- **"DORA Deployment Frequency vs Release Frequency?"** — Deployment = code goes to production. Release = feature available to users. With feature flags, you can deploy continuously but release on demand — highest maturity.

ARCHITECTURE BOUNDARY



37.8 One-Page SQL Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
Execution Order	FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT	You CANNOT reference a SELECT alias in WHERE (alias not yet defined at WHERE stage)
INNER JOIN	Returns rows where key exists in BOTH tables	Rows without a match in either table are excluded — use LEFT JOIN to preserve left table rows
LEFT JOIN	Returns ALL rows from left + matching from right (NULL if no match)	Always filter on RIGHT table's columns in WHERE or it silently converts to INNER JOIN
HAVING	Filters after GROUP BY (on aggregates)	HAVING cannot reference column aliases from SELECT ; use the aggregate expression directly
Window Functions	Compute across a set of rows related to current row; does NOT collapse rows	Unlike GROUP BY , original row count preserved; PARTITION BY defines the window frame
CTE	Named temporary result set defined before main query with WITH	CTE is re-evaluated per reference in most DBs (not materialized by default); use MATERIALIZED hint in PostgreSQL
DENSE_RANK()	Assigns rank with no gaps; ties share same rank	RANK() leaves gaps after ties (1,1,3); DENSE_RANK() does not (1,1,2); ROW_NUMBER() always unique
Anti-join	LEFT JOIN ... WHERE right.id IS NULL — find rows in A NOT in B	NOT IN with NULLs is a trap: NOT IN (1,2,NULL) returns 0 rows (NULL comparison unknown); use NOT EXISTS or LEFT JOIN
Index	B-tree structure; speeds up lookups/range scans; maintained on writes	Composite index (a, b, c) can only be used left-to-right: queries on (a), (a,b), (a,b,c) — not (b,c) alone
EXPLAIN / EXPLAIN ANALYZE	Shows query execution plan (estimated vs actual)	EXPLAIN shows plan; EXPLAIN ANALYZE actually executes — never ANALYZE on write queries in prod
NULL behavior	NULL is unknown; NULL = NULL is false; use IS NULL	NULL in NOT IN list causes entire NOT IN to return false for all rows
Isolation levels	READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE	Default in MySQL: REPEATABLE READ; PostgreSQL: READ COMMITTED. Higher isolation = more locking = less throughput

Concept	One-Line Definition	Critical Trap
COALESCE	Returns first non-NULL value in argument list	ISNULL() (SQL Server) / IFNULL() (MySQL) are DB-specific; COALESCE is ANSI standard
Subquery vs JOIN	Subquery often less optimizable; JOINS usually faster with indices	Correlated subquery runs once per outer row — $O(n^2)$; rewrite as JOIN for $O(n \log n)$
Transactions	ACID: Atomicity, Consistency, Isolation, Durability	COMMIT / ROLLBACK — without explicit transaction, each DML auto-commits (losing rollback option)

SQL EXECUTION ORDER (CRITICAL)

```
-- Logical execution order (not written order):
-- 1. FROM      - identify source tables
-- 2. JOIN       - combine tables on join conditions
-- 3. WHERE      - filter rows (before aggregation)
-- 4. GROUP BY   - group remaining rows
-- 5. HAVING     - filter groups (on aggregates)
-- 6. SELECT     - compute output columns & aliases
-- 7. DISTINCT   - remove duplicate result rows
-- 8. ORDER BY   - sort (can reference SELECT aliases here - they exist now)
-- 9. LIMIT      - cut output

-- This means:
SELECT department, COUNT(*) AS emp_count -- Step 6
FROM employees                          -- Step 1
WHERE hire_date > '2020-01-01'          -- Step 3 (CANNOT use emp_count here)
GROUP BY department                     -- Step 4
HAVING COUNT(*) > 5                     -- Step 5 (use COUNT(*), not emp_count)
ORDER BY emp_count DESC                  -- Step 8 (CAN use alias)
LIMIT 10;                               -- Step 9
```

CRITICAL CODE PATTERNS

```
-- 1. Window Functions - TOP-N per group with DENSE_RANK
WITH ranked_employees AS (
  SELECT
    e.name,
    e.department,
    e.salary,
    DENSE_RANK() OVER (
      PARTITION BY e.department
      ORDER BY e.salary DESC
    ) AS salary_rank
  FROM employees e
)
SELECT name, department, salary, salary_rank
FROM ranked_employees
WHERE salary_rank <= 3 -- top 3 earners per department
ORDER BY department, salary_rank;

-- 2. Anti-Join with LEFT JOIN (safe - handles NULLs)
-- Find customers who have NEVER placed an order
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.customer_id IS NULL; -- no matching order

-- WRONG approach (breaks with NULLs in subquery):
-- SELECT id FROM customers WHERE id NOT IN (SELECT customer_id FROM orders)
-- If ANY customer_id in orders is NULL → returns 0 rows!

-- Correct alternative using NOT EXISTS:
SELECT c.id, c.name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

```

-- 3. Running total (window function)
SELECT
    order_date,
    amount,
    SUM(amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
    AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg_7day
FROM daily_sales;

-- 4. CTE for complex multi-step query
WITH
active_users AS (
    SELECT user_id FROM users WHERE status = 'ACTIVE'
),
user_orders AS (
    SELECT o.user_id, COUNT(*) AS order_count, SUM(o.total) AS lifetime_value
    FROM orders o
    INNER JOIN active_users au ON o.user_id = au.user_id
    GROUP BY o.user_id
),
percentiles AS (
    SELECT
        user_id,
        lifetime_value,
        NTILE(4) OVER (ORDER BY lifetime_value DESC) AS quartile
    FROM user_orders
)
SELECT
    CASE quartile
        WHEN 1 THEN 'VIP'
        WHEN 2 THEN 'HIGH'
        WHEN 3 THEN 'MEDIUM'
        ELSE 'LOW'
    END AS segment,
    COUNT(*) AS user_count,
    AVG(lifetime_value) AS avg_value
FROM percentiles
GROUP BY quartile
ORDER BY quartile;

-- 5. Pivot / conditional aggregation
SELECT
    department,
    COUNT(CASE WHEN gender = 'M' THEN 1 END) AS male_count,
    COUNT(CASE WHEN gender = 'F' THEN 1 END) AS female_count,
    AVG(CASE WHEN gender = 'M' THEN salary END) AS avg_male_salary,
    AVG(CASE WHEN gender = 'F' THEN salary END) AS avg_female_salary
FROM employees
GROUP BY department;

-- 6. Duplicate detection and deduplication
-- Find duplicates:
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;

-- Delete duplicates, keep lowest ID:
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id) FROM users GROUP BY email
);

```

COMPARISON TABLES

Window Function	Resets Ties?	Gaps?	Use For
<code>ROW_NUMBER()</code>	No	N/A	Unique sequential ID
<code>RANK()</code>	Yes (shared rank)	YES (1,1,3)	Standard ranking with gaps
<code>DENSE_RANK()</code>	Yes (shared rank)	NO (1,1,2)	Ranking without gaps

Window Function	Resets Ties?	Gaps?	Use For
NTILE(n)	N/A	N/A	Quartile/percentile grouping
LAG(col, n)	N/A	N/A	Access previous row value
LEAD(col, n)	N/A	N/A	Access next row value

JOIN Type	Left rows	Right rows	NULL fill
INNER	matched only	matched only	No
LEFT	all	matched only	Right side
RIGHT	matched only	all	Left side
FULL OUTER	all	all	Both sides
CROSS	all	all (cartesian)	No

INTERVIEW TRAP ANSWERS

- **"Why does NOT IN fail with NULLs?"** — SQL uses three-valued logic (TRUE/FALSE/UNKNOWN). `x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. `x <> NULL` is UNKNOWN. `TRUE AND UNKNOWN = UNKNOWN`. No row passes an UNKNOWN predicate. Entire query returns 0 rows.
- **"Can you use a SELECT alias in WHERE?"** — No. WHERE executes before SELECT in logical order. The alias doesn't exist yet. Use a subquery or CTE to alias first, then filter.
- **"HAVING vs WHERE — when each?"** — WHERE filters individual rows BEFORE grouping (faster; leverages indexes). HAVING filters groups AFTER GROUP BY runs (on aggregate results). WHERE `COUNT(*) > 5` is illegal; must be HAVING `COUNT(*) > 5`.
- **"CTE vs Subquery — which is better?"** — CTEs improve readability and allow recursion. Performance-wise, most optimizers treat non-recursive CTEs identically to subqueries. PostgreSQL 12+ materializes CTEs by default (use NOT MATERIALIZED for inline). SQL Server inlines CTEs.
- **"What is a covering index?"** — An index that contains ALL columns needed by a query (in SELECT, WHERE, ORDER BY), so the engine never reads the table (heap). Maximum performance for specific query patterns.

ARCHITECTURE BOUNDARY

SQL Query Lifecycle:

Parser → Binder → Optimizer → Execution Engine → Storage

Query Optimizer considers:

- └ Statistics (row counts, cardinality, histograms)
- └ Available Indexes (B-tree, Hash, GIN, BRIN)
- └ Join order (reordered by optimizer for efficiency)
- └ Execution plan: Seq Scan / Index Scan / Index Only Scan
- └ Nested Loop / Hash Join / Merge Join
- └ Cost model (I/O + CPU estimates)

Window Function Frame Modes:

- └ ROWS BETWEEN X PRECEDING AND Y FOLLOWING (physical rows)
- └ RANGE BETWEEN X PRECEDING AND Y FOLLOWING (logical range by value)

37.9 One-Page Framework Architecture Sheet

KEY CONCEPTS

Concept	One-Line Definition	Critical Trap
Page Object Model	Encapsulates UI interactions in page classes; tests only call page methods	Pages should NOT contain assertions — put assertions in tests or separate assertion classes
Singleton Driver	One WebDriver instance per test class	NOT thread-safe for parallel execution — must be ThreadLocal<WebDriver> in parallel mode

Concept	One-Line Definition	Critical Trap
Test Data Strategy	External test data (JSON/CSV/DB) separates data from logic	Hardcoded test data = brittle tests; use <code>@DataProvider</code> or TestNG <code>@Parameters</code>
Reporting	Allure/Extent reports capture steps, screenshots, logs per test	Always attach screenshots in <code>@AfterMethod</code> on failure; Allure needs <code>allure-results</code> dir
Configuration	<code>config.properties</code> or <code>application.yml</code> externalizes env-specific values	Never commit credentials; use env vars or Vault; different <code>config-{env}.properties</code> per environment
Retry Mechanism	<code>IRetryAnalyzer</code> (TestNG) or <code>retries</code> (Playwright config) auto-retries flaky tests	Log retry attempts; alert if test retries consistently — indicates flaky test needing fix
Logging	SLF4J + Logback structured logging with MDC for test context	Use <code>MDC.put("testName", name)</code> for correlation across log statements in parallel runs
Utility Layer	Reusable helpers: <code>WaitUtils</code> , <code>ScreenshotUtils</code> , <code>DateUtils</code> , <code>FileUtils</code>	Utils should be stateless (static methods or pure functions) — no instance state
DriverFactory	Abstracts browser creation; supports local, remote (Selenium Grid), cloud (BrowserStack)	Use factory pattern with <code>@BeforeMethod</code> + <code>ThreadLocal</code> ; never <code>new ChromeDriver()</code> inline in test
API Client Layer	REST Assured client classes per API domain; separate from UI layer	API clients should be independent of test framework — pure Java/TypeScript for reuse
Assertion Layer	Wrapper around TestNG/JUnit asserts + custom messages	Add context to failures: <code>"Expected: dashboard URL but was: login URL [session expired?]"</code>
Environment Config	<code>ENV=staging mvn test</code> selects config via system property	<code>System.getProperty("env", "staging")</code> with fallback default — never hardcode env
Test Tags/Groups	TestNG <code>groups</code> / JUnit <code>@Tag</code> / Playwright <code>grep</code> — filter test execution	Tag consistency: <code>@smoke</code> , <code>@regression</code> , <code>@sanity</code> , <code>@critical</code> — enforce via PR checklist
Test Lifecycle	Setup → Execute → Assert → Teardown — every test must teardown even on failure	<code>@AfterMethod(alwaysRun = true)</code> ensures teardown runs even when test throws
Parallel Safety	<code>ThreadLocal</code> , immutable shared state, no static mutable fields	Static <code>WebDriver</code> in parallel = browsers interfere; static <code>count++</code> = race condition

CRITICAL CODE PATTERNS

```
// 1. Enterprise-grade DriverFactory
public class DriverFactory {
    private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return DRIVER.get();
    }

    public static void createDriver(String browser, boolean isRemote) {
        WebDriver driver;
        if (isRemote) {
            driver = createRemoteDriver(browser);
        } else {
            driver = createLocalDriver(browser);
        }
        driver.manage().timeouts().implicitlyWait(Duration.ZERO);
        driver.manage().window().maximize();
        DRIVER.set(driver);
    }

    private static WebDriver createLocalDriver(String browser) {
        return switch (browser.toLowerCase()) {
            case "chrome" -> {
                ChromeOptions opts = new ChromeOptions();
                opts.addArguments("--no-sandbox", "--disable-dev-shm-usage");
                if (Boolean.parseBoolean(System.getProperty("headless", "false"))) {
                    opts.addArguments("--headless=new");
                }
            }
        };
    }
}
```

```

        }
        yield new ChromeDriver(opts);
    }
    case "firefox" -> new FirefoxDriver(new FirefoxOptions());
    case "edge" -> new EdgeDriver(new EdgeOptions());
    default -> throw new IllegalArgumentException("Unsupported browser: " + browser);
};
}

private static WebDriver createRemoteDriver(String browser) {
    try {
        ChromeOptions opts = new ChromeOptions();
        return new RemoteWebDriver(
            new URL(System.getProperty("grid.url", "http://localhost:4444")), opts
        );
    } catch (MalformedURLException e) {
        throw new RuntimeException("Invalid Grid URL", e);
    }
}

public static void quitDriver() {
    if (DRIVER.get() != null) {
        DRIVER.get().quit();
        DRIVER.remove();
    }
}
}

// 2. Base test with full lifecycle
@Listeners({ScreenshotListener.class, AllureListener.class})
public abstract class BaseTest {
    protected static final Logger log = LoggerFactory.getLogger(BaseTest.class);

    @BeforeMethod(alwaysRun = true)
    public void setUp(Method method) {
        String browser = System.getProperty("browser", "chrome");
        boolean remote = Boolean.parseBoolean(System.getProperty("remote", "false"));
        MDC.put("testName", method.getName());
        log.info("Starting test: {}", method.getName());
        DriverFactory.createDriver(browser, remote);
        DriverFactory.getDriver().get(ConfigManager.getBaseUrl());
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown(ITestResult result) {
        log.info("Test {} - Status: {}", result.getName(), result.getStatus());
        DriverFactory.quitDriver();
        MDC.clear();
    }
}

// 3. Config manager (singleton with env support)
public class ConfigManager {
    private static final Properties props = new Properties();

    static {
        String env = System.getProperty("env", "staging");
        try (InputStream is = ConfigManager.class
            .getResourceAsStream("/config-" + env + ".properties")) {
            if (is == null) throw new RuntimeException("Config not found for env: " + env);
            props.load(is);
        } catch (IOException e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    public static String getBaseUrl() { return props.getProperty("base.url"); }
    public static String getApiBaseUrl() { return props.getProperty("api.base.url"); }
    public static int getDefaultTimeout() {
        return Integer.parseInt(props.getProperty("timeout.default", "15"));
    }
}

// 4. Framework layer diagram in code comments

```

```

/*
 * Test Layer      (tests/*.java)      - WHAT to test
 *   ↓
 * Page Layer      (pages/*.java)      - HOW to interact
 *   ↓
 * Component Layer  (components/*.java) - Reusable UI components
 *   ↓
 * Driver Layer     (DriverFactory.java) - Browser management
 *   ↓
 * Utility Layer     (utils/*.java)     - Cross-cutting helpers
 *   ↓
 * Config Layer     (ConfigManager.java) - Environment abstraction
 */

```

COMPARISON TABLES

Pattern	Benefit	When to Use
POM (Page Object)	Encapsulation, single place to update locators	Any UI test suite > 5 tests
Screenplay	Actor-based, highly composable, low coupling	Complex multi-actor user journey tests
DriverFactory + ThreadLocal	Thread-safe parallel execution	Any parallel test execution
Data-Driven	Separates test logic from data	Same logic, multiple data sets
Keyword-Driven	Non-technical stakeholders write tests	Rare; maintenance overhead high
Hybrid	POM + Data-driven + Config	Production enterprise frameworks

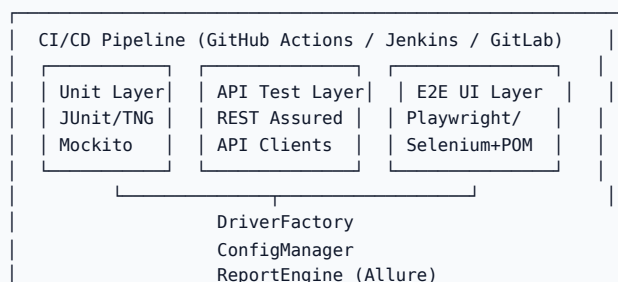
Layer	Responsibility	Anti-Pattern
Test	Assertions, test flow	Business logic, waits
Page	Locators, interactions	Assertions, data processing
Utility	Helpers (wait, screenshot)	Browser state, test logic
Config	Env values	Hardcoded values
Driver	Browser lifecycle	Test logic, data

INTERVIEW TRAP ANSWERS

- **"Why not put assertions in Page Objects?"** — Pages should be action libraries (HOW). Tests should express intent (WHAT and whether it worked). Assertions in pages make pages opinionated and hard to reuse across tests with different expected states.
- **"How do you handle shared test data between parallel tests?"** — Never share mutable test data. Use `@DataProvider` to give each thread its own data set. For DB data: use separate data sets per worker, or create+delete in setup/teardown with unique identifiers (UUID prefix).
- **"How do you ensure test independence?"** — Each test creates its own state, cleans up after itself. No test should depend on another test's side effects. Use `@BeforeMethod` for fresh state, not `@BeforeClass` shared state.
- **"How would you migrate a 500-test Selenium suite to Playwright?"** — Incremental: keep Selenium running, add Playwright for new tests. Identify highest-value/flakiest tests first. Extract page objects (if clean POM, only locator syntax changes). Run both in parallel CI until migration complete.

ARCHITECTURE BOUNDARY

Enterprise SDET Framework Architecture





37.10 One-Page Senior SDET Scenarios

KEY CONCEPTS

Scenario Type	What Interviewers Assess	Senior Differentiator
"Design a framework"	Architecture thinking, scalability, maintainability	Layered architecture, config management, CI integration, reporting, retry, parallel
"We have 40% flaky tests"	Debugging process, systematic root cause analysis	Quarantine → categorize (network/timing/data/env) → fix root cause, not symptom
"How would you test a REST API?"	Coverage breadth, contract testing awareness, security	Happy path + edge + boundary + schema + auth + rate-limit + performance + contract (Pact)
"How do you test a login feature?"	End-to-end thinking, negative testing, security	Valid, invalid, locked, SQL injection, XSS, brute-force, SSO, MFA, session timeout, concurrent
"Non-functional testing"	Performance, security, accessibility awareness	JMeter for load; OWASP ZAP for DAST; axe-core for a11y; Lighthouse for web vitals
"How do you shift-left?"	Prevention > detection mindset	Unit tests + mutation testing + contract tests + SAST + PR-level checks before merge
"Metrics to track quality"	Data-driven quality mindset	Defect escape rate, flaky test %, code coverage, DORA metrics, MTTR, test execution time
"What's in your smoke suite?"	Understanding of risk-based testing	Critical user journeys only (login, checkout, key API), runs in <5 minutes, blocks deploy
"How do you handle test data?"	Data management sophistication	Factory pattern, reset scripts, dedicated test data environments, synthetic data generation
"How do you test microservices?"	Distributed system test thinking	Contract tests (Pact), component tests with stubs, chaos engineering, observability

CRITICAL CODE PATTERNS

```
// 1. Testing API contract with Pact (consumer-driven contract testing)
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "UserService")
class UserApiContractTest {

    @Pact(consumer = "DashboardApp")
    RequestResponsePact getUserPact(PactDslWithProvider builder) {
        return builder
            .given("user 42 exists")
            .uponReceiving("GET user by ID")
                .path("/api/users/42")
                .method("GET")
                .headers(Map.of("Accept", "application/json"))
            .willRespondWith()
                .status(200)
                .body(new PactDslJsonBody()
                    .integerType("id", 42)
                    .stringType("name", "John Doe")
                    .stringType("email", "john@example.com"))
            .toPact();
    }

    @Test
    @PactTestFor(pactMethod = "getUserPact")
```

```

void testGetUser(MockServer mockServer) {
    UserApiClient client = new UserApiClient(mockServer.getUrl());
    User user = client.getUserById(42);
    assertThat(user.getId()).isEqualTo(42);
    assertThat(user.getName()).isNotEmpty();
}
}

// 2. Test data factory pattern
public class UserFactory {
    private static final Faker faker = new Faker();

    public static User validUser() {
        return User.builder()
            .name(faker.name().fullName())
            .email(faker.internet().emailAddress())
            .password("Valid@" + faker.number().digits(6))
            .role(Role.USER)
            .build();
    }

    public static User adminUser() {
        return validUser().toBuilder().role(Role.ADMIN).build();
    }

    public static User userWithInvalidEmail() {
        return validUser().toBuilder().email("not-an-email").build();
    }

    // DB factory – creates real record, returns for cleanup
    public static User persistedUser(UserRepository repo) {
        User user = validUser();
        User saved = repo.save(user);
        TestCleanupRegistry.register(() -> repo.delete(saved));
        return saved;
    }
}

// 3. Performance baseline test with JMeter DSL
TestPlan testPlan = TestPlanStats.testPlan(
    threadGroup(50, Duration.ofMinutes(5),
        httpSampler("Login", "https://api.staging.example.com/auth/login")
            .post(loginPayload, ContentType.APPLICATION_JSON)
        )
);
TestPlanStats stats = testPlan.run();
assertThat(stats.overall().sampleTimePercentile99()).isLessThan(Duration.ofSeconds(2));
assertThat(stats.overall().errorsCount()).isZero();

// 4. Accessibility test with axe-core (Selenium)
AxeBuilder axeBuilder = new AxeBuilder()
    .withTags(List.of("wcag2a", "wcag2aa"))
    .exclude(List.of(".third-party-widget"));

Results axeResults = axeBuilder.analyze(driver);
assertThat(axeResults.getViolations())
    .as("Accessibility violations found")
    .isEmpty();

```

SCENARIO WALKTHROUGHS

Scenario: "40% of our E2E tests are flaky. How do you fix this?"

Step 1 – MEASURE

- └─ Tag every flaky test (CI tracks pass/fail rate per test ID)
- └─ Categorize: network (30%) | timing (40%) | data (20%) | env (10%)
- └─ Build flakiness dashboard (% flaky, # retries, cost in CI minutes)

Step 2 – QUARANTINE

- └─ Move flaky tests to @flaky group (non-blocking CI)
- └─ Alert channel: "Test X flaked N times this week"
- └─ SLA: fix within 5 business days or delete

Step 3 – ROOT CAUSE PER CATEGORY

- └─ Timing: Replace Thread.sleep() → explicit waits; add stability checks
- └─ Network: Mock external dependencies; increase timeout for slow APIs
- └─ Data: Use factory-generated unique data; add cleanup in @AfterMethod
- └─ Environment: Pin Docker image versions; use dedicated test containers

Step 4 – PREVENT RECURRENCE

- └─ PR gate: new tests must pass 10 consecutive times before merge
- └─ Mandatory data-testid attributes for new UI components
- └─ Architecture review: shared state, ThreadLocal usage audit

Scenario: "Design an E2E test framework for a fintech app"**Requirements Analysis:**

- └─ Multi-browser (Chrome, Safari, Mobile)
- └─ API + UI testing
- └─ Parallel execution (fast feedback in CI)
- └─ Sensitive data (PCI compliance – no data in logs/screenshots)
- └─ Multiple environments (dev, staging, UAT, prod-smoke)
- └─ Cross-team contribution (junior + senior SDETs)

Solution:

- └─ Playwright (TypeScript) – E2E UI
- └─ REST Assured (Java) – API layer (or Playwright APIRequestContext)
- └─ Page Object Model + Fixtures (DI)
- └─ Allure Reports (screenshots redacted for PCI data)
- └─ Vault/AWS Secrets Manager → ConfigManager
- └─ GitHub Actions: smoke (5min) → regression (30min, parallel 8 workers)
- └─ Pact contract tests between microservices

INTERVIEW TRAP ANSWERS

- **"Why not automate everything?"** — Automation ROI diminishes for: one-off exploratory scenarios, highly volatile UI (changes weekly), complex visual verification, areas better served by unit tests. Automate stable, repeatable, high-value, high-frequency scenarios.
- **"What's the difference between verification and validation?"** — Verification: Are we building the product RIGHT? (meets spec, reviews, static analysis). Validation: Are we building the RIGHT product? (meets user needs, UAT, exploratory). SDET contributes to both.
- **"How do you know when a test suite is 'good enough'?"** — Risk-based: critical paths covered at all layers, defect escape rate trending down, coverage meets agreed threshold (e.g., 80% unit + 100% smoke + key regression paths), execution time within CI budget.
- **"How do you get developers to write more unit tests?"** — Shift-left culture: make unit test writing frictionless (templates, test generators), enforce coverage gates in CI, pair with devs to write first test together, show defect-escaped stats to build urgency, celebrate high-coverage PRs.
- **"What is test pyramid vs test trophy?"** — Pyramid: many unit, some integration, few E2E. Trophy (Kent C. Dodds): integration tests at the core (test behavior, not implementation), supported by unit and E2E. Trophy emphasizes user-centric integration tests over unit tests for UI components.

ARCHITECTURE BOUNDARY**Test Strategy Map (Risk-Based)**

Layer	Tool	Goal	CI Stage
Unit	JUnit + Mockito	Logic units	Stage 3
API Contract	Pact	Service compat	Stage 4
Component	REST Assured	API behavior	Stage 4
E2E Smoke	Playwright	Critical path	Stage 5
E2E Regression	Playwright	Full coverage	Stage 5
Performance	JMeter / k6	SLAs	Stage 6
Security	OWASP ZAP	Vulnerabilities	Stage 6
Accessibility	axe-core	WCAG 2.1 AA	Stage 5

37.11 Last 24-Hour Revision Checklist**T-24 HOURS: STRATEGIC REVIEW (Evening Before)**

- 18:00 - 19:00 | JAVA CORE (37.1 + 37.2)
- | ☒ Re-read volatile + synchronized + ThreadLocal
 - | ☒ Trace through double-checked locking mentally

	✓ Recite HashMap internal mechanics out loud ✓ Run through Collections comparison table
19:00 - 19:45	FRAMEWORK DESIGN (37.9) ✓ Sketch 4-layer POM architecture from memory ✓ Recite ThreadLocal DriverFactory pattern ✓ Explain parallel safety (what breaks, how to fix) ✓ Walk through config-per-environment pattern
19:45 - 20:30	SELENIUM + PLAYWRIGHT (37.3 + 37.6) ✓ Write explicit wait + fluent wait from memory ✓ Explain auto-waiting mechanics in Playwright ✓ Compare locator strategies (id > css > xpath) ✓ Page Object Model code pattern
20:30 - 21:00	REST ASSURED + API TESTING (37.5) ✓ Write given/when/then scaffold ✓ Explain RequestSpecification reuse ✓ JSON schema validation pattern ✓ Auth methods comparison
21:00 - 21:30	SQL (37.8) ✓ Recite execution order FROM-WHERE-GROUP-HAVING-SELECT-ORDER-LIMIT ✓ Write DENSE_RANK() top-3 per group from memory ✓ Write anti-join with LEFT JOIN WHERE NULL ✓ Explain NULL trap in NOT IN
21:30 - 22:00	CI/CD (37.7) ✓ Recite 7 stages of pipeline ✓ Blue-green vs canary vs rolling comparison ✓ DORA metrics: name all 4, know elite thresholds ✓ Write GHA YAML snippet from memory
22:00 - 22:30	TestNG (37.4) + SCENARIOS (37.10) ✓ @BeforeMethod vs @BeforeClass scope differences ✓ DataProvider vs Factory distinction ✓ SoftAssert assertAll() requirement ✓ Walk through "flaky test root cause" scenario
22:30 - 23:00	WILDCARD / WEAK AREA REVIEW ✓ Review any topic you hesitated on today ✓ Re-read 1-2 Interview Trap Answers per section ✓ Write out 3 STAR stories (Situation-Task-Action-Result) - Framework you built from scratch - Flaky test crisis you resolved - CI/CD improvement you led
23:00	STOP STUDYING. Hard stop. ✓ Prepare clothes, bag, documents ✓ Charge laptop and phone ✓ Write down interview location / dial-in details ✓ Set TWO alarms SLEEP – cognitive function degrades measurably without it

T-8 HOURS: WHAT TO SKIP

- ✗ DO NOT learn new concepts the morning of the interview
- ✗ DO NOT read new Stack Overflow threads
- ✗ DO NOT attempt new code challenges
- ✗ DO NOT read through entire notes end-to-end
- ✗ DO NOT over-caffeinate (spikes then crash)
- ✗ DO NOT arrive at interview location >30min early (increases anxiety)

T-4 HOURS: MORNING ROUTINE

Wake	✓ Drink water before coffee ✓ Light physical activity (20 min walk/stretch) – increases blood flow to prefrontal cortex
T-3.5h	✓ Light breakfast (complex carbs + protein, no heavy meals) ✓ Review ONLY the one-line definitions in 37.1-37.10 (30 min max)

	✓ Recite your 3 STAR stories out loud once each
T-2h	✓ Shower, dress (professional – Deloitte is client-facing culture) ✓ Check interview format: Technical screen? Design? Coding? Panel? ✓ Prepare questions TO ASK interviewer (write 5, ask 2-3)
T-1h	✓ Travel / log in to video call ✓ Test audio/video for remote; IDE ready for live coding ✓ Have water at your desk ✓ Close Slack, email, Discord – no notifications during interview

IN-INTERVIEW TACTICAL CHECKLIST

How to open every technical question:

1. CLARIFY FIRST – "Before I answer, can I confirm I understand the question?"
"Are we talking about parallel execution or sequential?"
2. STRUCTURE OUT LOUD – "I'll cover X, then Y, then Z"
3. CONCRETE THEN ABSTRACT – Give code/example first, then explain theory
4. TRADEOFFS – "The advantage of X is... the downside is... I'd choose X when..."
5. SENIOR SIGNAL – Mention monitoring, maintainability, CI integration, team impact

When you're stuck:

- ✓ "Let me think through this systematically..." (say it, then think)
- ✓ "I know this involves [related concept] – working from there..."
- ✓ "My instinct is X, let me reason through why..."
- ✓ "I'm not 100% certain on the exact syntax, but the approach is..."
- ✗ DO NOT say "I don't know" and stop – partial answers score points
- ✗ DO NOT guess silently – show your reasoning even if wrong

Behavioral question formula (STAR+):

Situation → Task → Action → Result → LEARNING (what you'd do differently)

The "Learning" step separates senior from mid-level engineers.
Senior engineers reflect and improve; mid-level engineers just report outcomes.

Code interview tactics:

- ✓ Think aloud – describe what you're writing before you write it
- ✓ Write the method signature first (shows design intent)
- ✓ Add comments for complex logic BEFORE code (proves you know what you're doing)
- ✓ Write happy path → then edge cases → then error handling
- ✓ Test with simple input mentally before declaring done
- ✓ Mention: "In production I'd also add [logging/retry/monitoring]"

Questions TO ASK interviewers:

1. "What does the test pyramid look like today on this team?"
2. "What are the biggest quality challenges you're trying to solve in the next 6 months?"
3. "How does the SDET team interact with developers during sprint planning?"
4. "What does the CI/CD pipeline look like – how long does a full pipeline run take?"
5. "What does success look like for this role in the first 90 days?"

Red flags to avoid:

- ✗ "We didn't have tests at my last job" – rephrase to "I drove adoption of testing"
- ✗ Criticizing previous employer's technology choices harshly
- ✗ "I usually just Google that" – even if true, say "I'd look up the syntax but understand..."
- ✗ Long silences without narration
- ✗ Over-explaining obvious points; under-explaining your reasoning

POST-INTERVIEW ACTIONS

Within 30 minutes:

- ✓ Write down every question asked (from memory, before it fades)
- ✓ Note what you answered well and what you struggled with
- ✓ Identify gaps → review those sections tonight

Within 24 hours:

- ✓ Send thank-you email to recruiter / interviewer if you have contact
Format: 3 sentences max – appreciation + one thing you found interesting from the interview + reaffirm interest

Within 48 hours:

- ✓ Update this notebook with any new questions encountered
- ✓ If you received a take-home assignment: read it immediately, clarify scope within 24h, deliver 48h early

If you get the offer:

- ✓ Negotiate – Deloitte salary bands have range; ask about:
 - Base salary + band level (Consultant / Senior Consultant / Manager)
 - Signing bonus
 - Certification sponsorship (ISTQB, AWS, etc.)
 - Remote work policy
 - Project staffing preferences (client site vs internal)

FINAL 60-SECOND CONFIDENCE BOOSTER

You have mastered:

- ✓ Java concurrency (volatile, synchronized, ThreadLocal, CompletableFuture)
- ✓ Collections internals (HashMap, ConcurrentHashMap, fail-fast/fail-safe)
- ✓ Selenium (explicit/fluent waits, POM, stale elements, parallel with ThreadLocal)
- ✓ TestNG (lifecycle, DataProvider vs Factory, SoftAssert, IRetryAnalyzer)
- ✓ REST Assured (BDD DSL, RequestSpec, schema validation, auth)
- ✓ Playwright (auto-wait, locators, fixtures, network mocking, TypeScript config)
- ✓ CI/CD (7-stage pipeline, deployment strategies, DORA metrics, GHA/Jenkins/GitLab)
- ✓ SQL (execution order, window functions, CTEs, anti-joins, TOP-N patterns)
- ✓ Framework Architecture (layered design, thread safety, config, reporting)
- ✓ Senior Scenarios (flaky tests, API design, performance, security, ally)

You are not just answering questions.

You are demonstrating how you THINK about quality at scale.

Show curiosity. Show depth. Show ownership.

Go get it.

End of Section 37 — Final Revision Sheets

Total Sections: 37.1 through 37.11

Cross-reference: Sections 1–36 for deep-dives on each topic